Job Intelligence Career Optimization Platform

1. Documentation scope and evidence

This document records the project state visible in the workspace on October 2, 2026. It is based on the files present in the project, not on assumptions about features that are not implemented.

The `docs` directory was empty before this document was created. No standalone log file was found in the project. Therefore, the "steps shown by the log" can only be documented from the executable source code and test scenarios currently present.

The project currently contains:

- `src/`: application source code.
- `tests/`: automated tests.
- `data/jobs.db`: SQLite database file.
- `.env.example`: required Adzuna credential names.
- `pyproject.toml`: setuptools build configuration and pytest settings.
- `README.md`: present but with no documented project instructions.
- `.venv/`: local Python environment and installed dependencies.
- `docs/`: this project record.

2. Current product purpose

The implemented portion of the Job Intelligence Career Optimization Platform collects job postings from an external job source, converts them into a common job format, validates and cleans their data, prevents duplicate records, and stores the results in SQLite for later inspection.

The current implementation is a job-intelligence ingestion foundation. It does not yet implement a candidate profile, resume parsing, skill-gap analysis, career recommendations, application tracking, ranking, a web interface, or an API server.

3. End-to-end workflow

The complete implemented workflow is:

1. Load Adzuna credentials from environment variables.
2. Construct an `AdzunaSource` with an application ID, application key, and optional country code.
3. Receive a search query and location.
4. Build an Adzuna URL for the selected country and page.
5. Send an HTTP GET request with credentials and search parameters.
6. Enforce an HTTP success response with `raise_for_status()`.
7. Decode the JSON response.
8. Read the response's `results` array, defaulting to an empty list when it is absent.
9. Map each Adzuna result into the platform's common job dictionary.
10. Normalize each dictionary:
    - Convert non-null values to strings.
    - Trim leading and trailing whitespace.
    - Collapse repeated internal whitespace.
    - Preserve null values as null.
    - Add a UTC collection timestamp when one was not supplied.
11. Validate the normalized dictionary as a `JobRecord`.
12. Reject records missing required values or violating field constraints.
13. Convert the validated `JobRecord` into a SQLAlchemy `Job` model.
14. Check for an existing record with the same source and source job ID.
15. Return the existing record for a duplicate, without inserting a second row.
16. Flush new records to the current SQLAlchemy session.
17. Commit the session after all fetched records have been processed.
18. Return an `IngestionResult` containing fetch, insert, duplicate, failure, and detailed error counts.
19. Use database inspection queries to print total jobs, source counts, top companies, and top locations.

4. Configuration and external source

4.1 Environment variables

`src/config.py` calls `load_dotenv()` and reads:

- `ADZUNA_APP_ID`
- `ADZUNA_APP_KEY`

Importing this module raises:

```
ValueError: Missing ADZUNA_APP_ID or ADZUNA_APP_KEY in .env
```

when either value is missing or empty. The checked-in `.env.example` contains the two variable names with blank values. Real credentials are expected to be provided through a local `.env` file or the process environment and must not be committed.

4.2 Adzuna request

`src/ingestion/adzuna.py` defines `AdzunaSource` with:

- Base URL: `https://api.adzuna.com/v1/api/jobs`
- Default country: `us`
- Default page: `1`
- Default results per page: `20`
- HTTP timeout: `30` seconds

For a query and location, the request path is:

```
{BASE_URL}/{country}/search/{page}
```

The request query parameters are:

- `app_id`
- `app_key`
- `what`: search query
- `where`: location
- `results_per_page`
- `content-type=application/json`

The source lets `requests` raise its normal HTTP exception for non-success responses. It does not add a retry policy, rate-limit handling, pagination loop, or custom network-error classification.

4.3 Adzuna-to-platform mapping

Each Adzuna result is mapped as follows:

Platform field	Adzuna source
`source`	Constant `"adzuna"`
`source_job_id`	`id`, converted to a string
`title`	`title`
`company`	`company.display_name`
`location`	`location.display_name`
`remote_type`	Not supplied; `None`
`employment_type`	Not supplied; `None`
`salary_min`	`salary_min`
`salary_max`	`salary_max`
`salary_currency`	Constant `"USD"`
`description`	`description`
`requirements`	Not supplied; `None`
`responsibilities`	Not supplied; `None`
`skills`	Not supplied; `None`
`job_url`	`redirect_url`
`posted_at`	`created`

The adapter expects each result to contain `id`; direct indexing means a result without `id` raises a `KeyError` during response normalization.

5. Common job contract and validation

`src/ingestion/base.py` defines the abstract `JobSource` contract. Sources must provide:

```
fetch_jobs(query: str, location: str) -> list[dict[str, Any]]
```

`src/processing/job_schema.py` defines the Pydantic `JobRecord`. Extra input keys are ignored.

5.1 Required fields

The following fields must be non-empty strings:

- `source`
- `title`
- `company`
- `job_url`

`collected_at` is also required and must be a datetime. The normalizer supplies it automatically when it is absent.

5.2 Optional fields

These values may be null:

- `source_job_id`
- `location`
- `remote_type`
- `employment_type`
- `salary_min`
- `salary_max`
- `salary_currency`
- `description`
- `requirements`
- `responsibilities`
- `skills`
- `posted_at`

Salary values are typed as optional floats. The source adapter passes through the source values and Pydantic performs the final validation/coercion.

5.3 Text normalization

`src/processing/normalizer.py` applies `normalize_text` to every raw input field. Non-null values are stringified, stripped, and whitespace-collapsed. Empty strings become `None`.

The collection timestamp is:

- The input `collected_at` value when supplied.
- Otherwise, the current UTC time with timezone information removed.

The resulting dictionary is validated with `JobRecord.model_validate`.

6. Persistence design

6.1 Database location and initialization

`src/database/connection.py` computes the project root from the source file and uses:

```
data/jobs.db
```

The SQLAlchemy URL is SQLite-based. The engine is created with `echo=False`. `SessionLocal` is configured with autoflush disabled and autocommit disabled.

`init_db()` creates tables using SQLAlchemy metadata. It does not delete or rebuild existing data.

6.2 `jobs` table

`src/database/models.py` defines the `Job` table:

Column	Type	Required/default
`id`	Integer	Primary key, autoincrement
`source`	String(100)	Required
`source_job_id`	String(255)	Optional
`title`	String(500)	Required
`company`	String(255)	Required
`location`	String(500)	Optional
`remote_type`	String(50)	Optional
`employment_type`	String(100)	Optional
`salary_min`	Float	Optional
`salary_max`	Float	Optional
`salary_currency`	String(10)	Optional
`description`	Text	Optional
`requirements`	Text	Optional
`responsibilities`	Text	Optional
`skills`	Text	Optional
`job_url`	String(1000)	Required
`posted_at`	DateTime	Optional
`collected_at`	DateTime	Required; UTC-naive `datetime.utcnow` default
`created_at`	DateTime	Required; UTC-naive `datetime.utcnow` default

There is no database-level unique constraint shown for `(source, source_job_id)`. Duplicate prevention is implemented in repository code.

6.3 Mapping

`src/database/mappers.py` explicitly copies every `JobRecord` field into a new SQLAlchemy `Job` model. It does not persist unknown fields.

6.4 Repository operations

`src/database/repository.py` provides:

- `get_by_source_job_id(source, source_job_id)`: finds a record by both source and source-specific ID.
- `add(job)`: adds and flushes a job.
- `get_all()`: returns jobs ordered by ascending database ID.
- `add_if_new(job)`: checks the composite source/source-ID identity when an ID exists; returns `(existing_job, False)` for duplicates and `(new_job, True)` after inserting a new record.

If `source_job_id` is null, `add_if_new` does not perform a duplicate lookup and allows insertion.

6.5 Database inspection

Running `src/database/inspect.py` opens a session and prints:

- Total job count.
- Counts grouped by source.
- The ten most frequent companies.
- The ten most frequent non-null locations.

The inspection function closes its session in a `finally` block.

7. Ingestion pipeline behavior

`src/ingestion/pipeline.py` defines two result structures.

7.1 `IngestionError`

Each failure records:

- `source_job_id`
- Exception class name in `error_type`
- Exception text in `message`

7.2 `IngestionResult`

The counters are:

- `fetched`: number of dictionaries returned by the source.
- `inserted`: number of new database rows.
- `duplicates`: number of existing rows reused.
- `failed`: number of records that failed normalization, validation, mapping, or repository processing.
- `errors`: list of detailed `IngestionError` entries.

The source fetch itself occurs before per-record error handling. A source-level HTTP or JSON failure therefore propagates out of `ingest_jobs` rather than being represented in `IngestionResult`.

Per-record exceptions are caught broadly, counted, and retained in the result. The pipeline continues processing later records. After the loop, it commits the session once.

8. Test and verification record

`pyproject.toml` configures pytest to:

- Add the project root to `pythonpath`.
- Discover tests under `tests`.

The test fixture in `tests/conftest.py` creates an isolated in-memory SQLite database, creates the metadata tables, yields a SQLAlchemy session, and disposes the engine afterward.

8.1 Contract test

`tests/test_ingestion.py` verifies that a fake source can satisfy the `JobSource` interface and return a job containing the expected title and source.

8.2 Normalization tests

`tests/test_ingestion_pipeline.py` verifies that:

- Leading/trailing whitespace is removed.
- Repeated internal spaces are collapsed.
- Optional fields can be absent.
- Missing salary and location values become `None`.

8.3 Source mapping and schema integration

`tests/test_pipeline.py` verifies that a representative Adzuna result:

- Becomes source `"adzuna"`.
- Converts numeric ID `98765` to string `"98765"`.
- Normalizes title, company, and location.
- Preserves salary bounds.
- Preserves the job URL.

8.4 Full ingestion pipeline tests

`tests/test_processing.py` verifies:

1. Two valid fake jobs are fetched and inserted.
2. Running the same source twice inserts two jobs on the first run and reports two duplicates on the second run.
3. An invalid title causes one validation failure while the valid job is still persisted.
4. The failure is reported as a Pydantic `ValidationError` with the original source job ID.

8.5 Repository tests

`tests/test_database.py` verifies:

- A job can be added, committed, and retrieved by source job ID.
- A second job with the same source and source job ID returns the first stored row and does not overwrite its title.

8.6 Adzuna adapter tests

`tests/test_adzuna.py` verifies response mapping for title, company, location, salary, currency, source ID, and URL.

8.7 Live tests

The following tests use real Adzuna credentials and network access:

- `tests/test_adzuna_live.py`: requests up to five Chicago data-scientist jobs and verifies a list response.
- `tests/test_live_pipeline.py`: fetches live jobs, normalizes them, and checks required values and reasonable posted-year values.
- `tests/test_live_ingestion.py`: initializes the project database, runs live ingestion, prints counters/errors, and verifies that fetched records equal inserted plus duplicate plus failed records.

Live tests are environment-dependent and can consume external API quota. They are distinct from the deterministic in-memory tests.

9. What is not implemented or not documented

The current workspace provides no verified implementation for:

- Candidate profile storage.
- Resume or CV ingestion.
- Resume-to-job matching.
- Skill extraction or skill taxonomy.
- Career-path optimization.
- Personalized recommendations.
- Job scoring or ranking.
- Application tracking.
- Notifications.
- Authentication or authorization.
- Frontend/UI.
- REST or GraphQL endpoints.
- Background scheduling.
- Multi-source ingestion beyond the Adzuna adapter.
- API retries, rate-limit backoff, or request caching.
- Database migrations.
- Database-level duplicate constraints.
- Structured logging.
- Production deployment.
- Observability or metrics.

The existing logging is limited to `print` output in the live ingestion test and database inspection utility. No application logging framework is present.

10. Known operational considerations

1. Credentials must be supplied before importing `src.config`.
2. Live tests require outbound network access and valid Adzuna credentials.
3. `data/jobs.db` is a persistent local database and should be backed up before destructive maintenance.
4. Duplicate detection occurs in application code rather than through a unique database constraint, so concurrent ingestion workers could require stronger database-level protection.
5. Source-level fetch failures are not converted into `IngestionResult` errors.
6. The Adzuna adapter currently labels all salary currency values as USD.
7. The adapter leaves remote type, employment type, requirements, responsibilities, and skills empty.
8. Datetimes are stored as naive values after UTC normalization for newly collected records; source-provided date parsing is delegated to Pydantic.
9. The default `JobSource` interface does not expose page size, although the Adzuna implementation adds optional pagination parameters.

11. Reproducible execution sequence

For a local deterministic verification run:

1. Use the project's Python environment.
2. Install the dependencies declared by the project environment, including pytest, SQLAlchemy, Pydantic, python-dotenv, and requests.
3. Run the deterministic tests under `tests` while excluding live tests when credentials or network access are unavailable.
4. Confirm that the in-memory database tests pass without modifying `data/jobs.db`.

For a live ingestion run:

1. Provide `ADZUNA_APP_ID` and `ADZUNA_APP_KEY`.
2. Import the configuration and source classes.
3. Call `init_db()`.
4. Create `AdzunaSource`.
5. Create a `SessionLocal` session.
6. Call `ingest_jobs` with a query and location.
7. Review `fetched`, `inserted`, `duplicates`, `failed`, and `errors`.
8. Close the session in a `finally` block.
9. Run the database inspection utility to review aggregate counts.

12. File-by-file index

Application source

- `src/config.py`: environment loading and credential validation.
- `src/ingestion/base.py`: abstract source interface.
- `src/ingestion/adzuna.py`: Adzuna HTTP adapter and response mapping.
- `src/ingestion/pipeline.py`: fetch, normalize, validate, deduplicate, persist, and summarize workflow.
- `src/processing/job_schema.py`: Pydantic common job contract.
- `src/processing/normalizer.py`: text cleanup and collection timestamp logic.
- `src/database/connection.py`: SQLite URL, SQLAlchemy engine/session, and table initialization.
- `src/database/models.py`: SQLAlchemy base and `jobs` table.
- `src/database/mappers.py`: Pydantic-to-SQLAlchemy conversion.
- `src/database/repository.py`: job lookup, insertion, listing, and duplicate prevention.
- `src/database/inspect.py`: printed database summary.
- Package `__init__.py` files: package markers; no additional behavior is documented in them.

Tests

- `tests/conftest.py`: isolated database fixture.
- `tests/test_ingestion.py`: source contract.
- `tests/test_ingestion_pipeline.py`: normalization and optional-field behavior.
- `tests/test_pipeline.py`: Adzuna-to-record integration.
- `tests/test_processing.py`: full ingestion pipeline behavior, including insertion, deduplication, and validation-error reporting.
- `tests/test_database.py`: repository persistence and duplicate handling.
- `tests/test_adzuna.py`: deterministic Adzuna response mapping.
- `tests/test_adzuna_live.py`: live source connectivity.
- `tests/test_live_pipeline.py`: live fetch and normalization.
- `tests/test_live_ingestion.py`: live persistence and result accounting.